

Basic Information

Title: Java Collection Framework – Important Methods

Source File: CollectionFrameworkDemo.java

Objective / Aim

To implement and understand the commonly used methods of the Java Collection Framework using List, Set, SortedSet, NavigableSet, Queue, Deque, Map, SortedMap, NavigableMap, Iterator, and ListIterator with simple examples.

Description

This program demonstrates how different Java collection interfaces are used to store, access, modify, search, sort, navigate, and traverse data. Each interface is demonstrated with a suitable implementation class such as ArrayList, HashSet, TreeSet, LinkedList, ArrayDeque, HashMap, and TreeMap.

System Requirements

Hardware: Standard PC/Laptop, minimum 4 GB RAM, keyboard and mouse.

Software: Windows/Linux/macOS, JDK 17 or JDK 21, Java compiler, and any Java IDE or Command Prompt/Terminal.

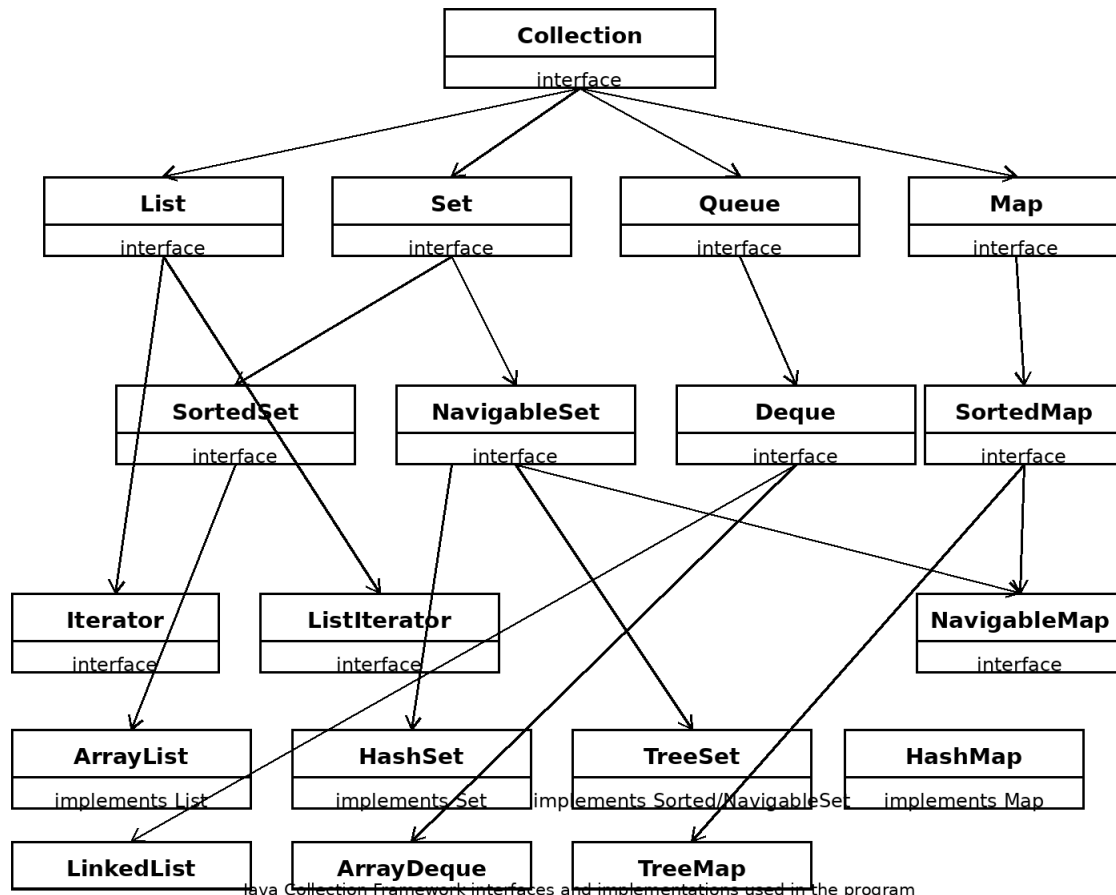
Algorithm / Logic Description

Create collection objects using the required Java interfaces and implementation classes. Perform insertion, removal, searching, size checking, sorting, range operations, navigation operations, queue/deque operations, map operations, and iterator traversal. Display the result after each group of operations.

Algorithm

1. Start the program.
2. Create a Scanner object and read two elements from the user.
3. Demonstrate Collection methods using ArrayList.
4. Demonstrate List methods such as add, get, set, remove, indexOf, subList, and sort.
5. Demonstrate Set methods using HashSet and show duplicate removal.
6. Demonstrate SortedSet and NavigableSet methods using TreeSet.
7. Demonstrate Queue methods using LinkedList.
8. Demonstrate Deque methods using ArrayDeque.
9. Demonstrate Map methods using HashMap.
10. Demonstrate SortedMap and NavigableMap methods using TreeMap.
11. Traverse a list using Iterator and demonstrate remove() and forEachRemaining().
12. Traverse a list in both directions using ListIterator and demonstrate add() and set().
13. Display the results of all operations.
14. Close the Scanner and stop the program.

UML Class / Interface Diagram



Source Code

```
import java.util.*;

public class CollectionFrameworkDemo {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        // ===== COLLECTION =====
        System.out.println("===== COLLECTION INTERFACE =====");

        ArrayList<String> collection = new ArrayList<>();
        ArrayList<String> extra = new ArrayList<>();

        System.out.print("Enter first element: ");
        String first = sc.nextLine();

        System.out.print("Enter second element: ");
        String second = sc.nextLine();

        collection.add(first);
        collection.add(second);

        extra.add("Java");
        extra.add("Python");

        collection.addAll(extra);

        System.out.println("Collection: " + collection);
        System.out.println("Size: " + collection.size());
        System.out.println("Contains Java: " + collection.contains("Java"));
        System.out.println("Contains all: " + collection.containsAll(extra));
        System.out.println("Is Empty: " + collection.isEmpty());
    }
}
```

Source Code (Continued)

```
collection.remove("Python");
collection.removeAll(extra);

System.out.println("After remove operations: " + collection);

collection.addAll(extra);
collection.clear();

System.out.println("After clear: " + collection);
System.out.println("Is Empty: " + collection.isEmpty());

// ===== LIST =====
System.out.println("\n===== LIST INTERFACE =====");

List<String> list = new ArrayList<>();

list.add("Java");
list.add("Python");
list.add("C");
list.add("Java");

list.add(1, "C++");

System.out.println("List: " + list);
System.out.println("Element at index 2: " + list.get(2));

list.set(2, "HTML");
System.out.println("After set: " + list);

list.remove(1);

System.out.println("First Java index: " + list.indexOf("Java"));
System.out.println("Last Java index: " + list.lastIndexOf("Java"));

System.out.println("Sub List: " + list.subList(0, 2));

list.sort(Comparator.naturalOrder());
System.out.println("Sorted List: " + list);

// ===== SET =====
System.out.println("\n===== SET INTERFACE =====");

Set<String> set = new HashSet<>();

set.add("Java");
set.add("Python");
set.add("Java");

System.out.println("Set: " + set);
System.out.println("Contains Java: " + set.contains("Java"));
System.out.println("Size: " + set.size());
System.out.println("Is Empty: " + set.isEmpty());

set.remove("Python");
System.out.println("After remove: " + set);

set.clear();
System.out.println("After clear: " + set);

// ===== SORTED SET =====
System.out.println("\n===== SORTED SET INTERFACE =====");

SortedSet<Integer> sortedSet = new TreeSet<>();

sortedSet.add(30);
sortedSet.add(10);
sortedSet.add(20);
sortedSet.add(40);

System.out.println("Sorted Set: " + sortedSet);
```

Source Code (Continued)

```
System.out.println("First: " + sortedSet.first());
System.out.println("Last: " + sortedSet.last());
System.out.println("Head Set: " + sortedSet.headSet(30));
System.out.println("Tail Set: " + sortedSet.tailSet(20));
System.out.println("Sub Set: " + sortedSet.subSet(20, 40));
System.out.println("Comparator: " + sortedSet.comparator());

// ===== NAVIGABLE SET =====
System.out.println("\n===== NAVIGABLE SET INTERFACE =====");

NavigableSet<Integer> navSet = new TreeSet<>();

navSet.add(10);
navSet.add(20);
navSet.add(30);
navSet.add(40);

System.out.println("Set: " + navSet);
System.out.println("Lower than 30: " + navSet.lower(30));
System.out.println("Floor of 30: " + navSet.floor(30));
System.out.println("Ceiling of 25: " + navSet.ceiling(25));
System.out.println("Higher than 30: " + navSet.higher(30));

System.out.println("Poll First: " + navSet.pollFirst());
System.out.println("Poll Last: " + navSet.pollLast());

System.out.println("Descending Set: " + navSet.descendingSet());

// ===== QUEUE =====
System.out.println("\n===== QUEUE INTERFACE =====");

Queue<String> queue = new LinkedList<>();

queue.add("A");
queue.offer("B");
queue.offer("C");

System.out.println("Queue: " + queue);
System.out.println("Element: " + queue.element());
System.out.println("Peek: " + queue.peek());

System.out.println("Remove: " + queue.remove());
System.out.println("Poll: " + queue.poll());

System.out.println("Queue after removal: " + queue);

// ===== DEQUE =====
System.out.println("\n===== DEQUE INTERFACE =====");

Deque<String> deque = new ArrayDeque<>();

deque.addFirst("B");
deque.addLast("C");
deque.offerFirst("A");
deque.offerLast("D");

System.out.println("Deque: " + deque);
System.out.println("Peek First: " + deque.peekFirst());
System.out.println("Peek Last: " + deque.peekLast());

System.out.println("Remove First: " + deque.removeFirst());
System.out.println("Remove Last: " + deque.removeLast());

System.out.println("Poll First: " + deque.pollFirst());
System.out.println("Poll Last: " + deque.pollLast());

// ===== MAP =====
System.out.println("\n===== MAP INTERFACE =====");

Map<Integer, String> map = new HashMap<>();
```

Source Code (Continued)

```
map.put(101, "Rahul");
map.put(102, "Priya");
map.put(103, "Arjun");

System.out.println("Map: " + map);
System.out.println("Value for 102: " + map.get(102));
System.out.println("Contains key 101: " + map.containsKey(101));
System.out.println("Contains value Rahul: "
    + map.containsValue("Rahul"));

System.out.println("Keys: " + map.keySet());
System.out.println("Values: " + map.values());
System.out.println("Entries: " + map.entrySet());
System.out.println("Size: " + map.size());
System.out.println("Is Empty: " + map.isEmpty());

map.remove(103);
System.out.println("After remove: " + map);

map.clear();
System.out.println("After clear: " + map);

// ===== SORTED MAP =====
System.out.println("\n===== SORTED MAP INTERFACE =====");

SortedMap<Integer, String> sortedMap = new TreeMap<>();

sortedMap.put(30, "C");
sortedMap.put(10, "A");
sortedMap.put(20, "B");
sortedMap.put(40, "D");

System.out.println("Sorted Map: " + sortedMap);
System.out.println("First Key: " + sortedMap.firstKey());
System.out.println("Last Key: " + sortedMap.lastKey());
System.out.println("Head Map: " + sortedMap.headMap(30));
System.out.println("Tail Map: " + sortedMap.tailMap(20));
System.out.println("Sub Map: " + sortedMap.subMap(20, 40));
System.out.println("Comparator: " + sortedMap.comparator());

// ===== NAVIGABLE MAP =====
System.out.println("\n===== NAVIGABLE MAP INTERFACE =====");

NavigableMap<Integer, String> navMap = new TreeMap<>();

navMap.put(10, "A");
navMap.put(20, "B");
navMap.put(30, "C");
navMap.put(40, "D");

System.out.println("Map: " + navMap);
System.out.println("Lower Key: " + navMap.lowerKey(30));
System.out.println("Floor Key: " + navMap.floorKey(30));
System.out.println("Ceiling Key: " + navMap.ceilingKey(25));
System.out.println("Higher Key: " + navMap.higherKey(30));

System.out.println("First Entry: " + navMap.firstEntry());
System.out.println("Last Entry: " + navMap.lastEntry());

System.out.println("Poll First Entry: " + navMap.pollFirstEntry());
System.out.println("Poll Last Entry: " + navMap.pollLastEntry());

System.out.println("Descending Map: " + navMap.descendingMap());

// ===== ITERATOR =====
System.out.println("\n===== ITERATOR INTERFACE =====");

List<String> languages = new ArrayList<>();
```

Source Code (Continued)

```
languages.add("Java");
languages.add("Python");
languages.add("C");

Iterator<String> iterator = languages.iterator();

while (iterator.hasNext()) {
    String value = iterator.next();
    System.out.println(value);
}

iterator = languages.iterator();

while (iterator.hasNext()) {
    if (iterator.next().equals("Python")) {
        iterator.remove();
    }
}

System.out.println("After Iterator remove: " + languages);

iterator = languages.iterator();

System.out.println("Remaining elements:");
iterator.forEachRemaining(System.out::println);

// ===== LIST ITERATOR =====
System.out.println("\n===== LIST ITERATOR INTERFACE =====");

List<String> subjects = new ArrayList<>();

subjects.add("Java");
subjects.add("DBMS");
subjects.add("CN");

ListIterator<String> listIterator =
    subjects.listIterator();

System.out.println("Forward Traversal:");

while (listIterator.hasNext()) {
    System.out.println(
        listIterator.nextIndex() + " : "
        + listIterator.next());
}

System.out.println("Backward Traversal:");

while (listIterator.hasPrevious()) {
    System.out.println(
        listIterator.previousIndex() + " : "
        + listIterator.previous());
}

listIterator = subjects.listIterator();

listIterator.next();
listIterator.set("Advanced Java");
listIterator.add("Python");

System.out.println("After add and set: " + subjects);

System.out.println("\n===== PROGRAM COMPLETED =====");

sc.close();
}
}
```

```
Element at index 2: Python
After set: [Java, C++, HTML, C, Java]
First Java index: 0
Last Java index: 3
Sub List: [Java, HTML]
Sorted List: [C, HTML, Java, Java]

===== SET INTERFACE =====
Set: [Java, Python]
Contains Java: true
Size: 2
Is Empty: false
After remove: [Java]
After clear: []

===== SORTED SET INTERFACE =====
Sorted Set: [10, 20, 30, 40]
First: 10
Last: 40
Head Set: [10, 20]
Tail Set: [20, 30, 40]
Sub Set: [20, 30]
Comparator: null

===== NAVIGABLE SET INTERFACE =====
Set: [10, 20, 30, 40]
Lower than 30: 20
Floor of 30: 30
Ceiling of 25: 30
Higher than 30: 40
Poll First: 10
Poll Last: 40
Descending Set: [30, 20]

===== QUEUE INTERFACE =====
Queue: [A, B, C]
Element: A
Peek: A
Remove: A
Poll: B
Queue after removal: [C]

===== DEQUE INTERFACE =====
Deque: [A, B, C, D]
Peek First: A
Peek Last: D
Remove First: A
Remove Last: D
Poll First: B
Poll Last: C

===== MAP INTERFACE =====
Map: {101=Rahul, 102=Priya, 103=Arjun}
Value for 102: Priya
Contains key 101: true
Contains value Rahul: true
Keys: [101, 102, 103]
Values: [Rahul, Priya, Arjun]
Entries: [101=Rahul, 102=Priya, 103=Arjun]
Size: 3
Is Empty: false
After remove: {101=Rahul, 102=Priya}
After clear: {}

===== SORTED MAP INTERFACE =====
Sorted Map: {10=A, 20=B, 30=C, 40=D}
First Key: 10
Last Key: 40
Head Map: {10=A, 20=B}
Tail Map: {20=B, 30=C, 40=D}
Sub Map: {20=B, 30=C}
Comparator: null

===== NAVIGABLE MAP INTERFACE =====
Map: {10=A, 20=B, 30=C, 40=D}
Lower Key: 20
Floor Key: 30
Ceiling Key: 30
Higher Key: 40
First Entry: 10=A
Last Entry: 40=D
Poll First Entry: 10=A
Poll Last Entry: 40=D
Descending Map: {30=C, 20=B}

===== ITERATOR INTERFACE =====
Java
Python
C
After Iterator remove: [Java, C]
Remaining elements:
Java
C

===== LIST ITERATOR INTERFACE =====
Forward Traversal:
0 : Java
1 : DBMS
2 : CN
Backward Traversal:
2 : CN
1 : DBMS
0 : Java
After add and set: [Advanced Java, Python, DBMS, CN]

===== PROGRAM COMPLETED =====
```

```

===== COLLECTION INTERFACE =====
Enter first element: Enter second element: Collection: [SQL, Python, Java, Python]
Size: 4
Contains Java: true
Contains all: true
Is Empty: false
After remove operations: [SQL]
After clear: []
Is Empty: true

===== LIST INTERFACE =====
List: [Java, C++, Python, C, Java]
Element at index 2: Python
After set: [Java, C++, HTML, C, Java]
First Java index: 0
Last Java index: 3
Sub List: [Java, HTML]
Sorted List: [C, HTML, Java, Java]

===== SET INTERFACE =====
Set: [Java, Python]
Contains Java: true
Size: 2
Is Empty: false
After remove: [Java]
After clear: []

===== SORTED SET INTERFACE =====
Sorted Set: [10, 20, 30, 40]
First: 10
Last: 40
Head Set: [10, 20]
Tail Set: [20, 30, 40]
Sub Set: [20, 30]
Comparator: null

===== NAVIGABLE SET INTERFACE =====
Set: [10, 20, 30, 40]
Lower than 30: 20
Floor of 30: 30
Ceiling of 25: 30
Higher than 30: 40
Poll First: 10
Poll Last: 40
Descending Set: [30, 20]

===== QUEUE INTERFACE =====
Queue: [A, B, C]
Element: A
Peek: A
Remove: A
Poll: B
Queue after removal: [C]

===== DEQUE INTERFACE =====
Deque: [A, B, C, D]
Peek First: A
Peek Last: D
Remove First: A
Remove Last: D
Poll First: B
Poll Last: C

===== MAP INTERFACE =====
Map: {101=Rahul, 102=Priya, 103=Arjun}
Value for 102: Priya
Contains key 101: true
Contains value Rahul: true
Keys: [101, 102, 103]
Values: [Rahul, Priya, Arjun]
Entries: [101=Rahul, 102=Priya, 103=Arjun]
Size: 3
Is Empty: false
After remove: {101=Rahul, 102=Priya}
After clear: {}

===== SORTED MAP INTERFACE =====
Sorted Map: {10=A, 20=B, 30=C, 40=D}
First Key: 10
Last Key: 40
Head Map: {10=A, 20=B}
Tail Map: {20=B, 30=C, 40=D}
Sub Map: {20=B, 30=C}
Comparator: null

===== NAVIGABLE MAP INTERFACE =====
Map: {10=A, 20=B, 30=C, 40=D}
Lower Key: 20
Floor Key: 30
Ceiling Key: 30
Higher Key: 40
First Entry: 10=A
Last Entry: 40=D
Poll First Entry: 10=A
Poll Last Entry: 40=D
Descending Map: {30=C, 20=B}

===== ITERATOR INTERFACE =====
Java
Python
C
After Iterator remove: [Java, C]
Remaining elements:
Java
C

===== LIST ITERATOR INTERFACE =====
Forward Traversal:
0 : Java
1 : DBMS
2 : CN
Backward Traversal:
2 : CN
1 : DBMS
0 : Java
After add and set: [Advanced Java, Python, DBMS, CN]

===== PROGRAM COMPLETED =====

```

Result

The Java Collection Framework program was successfully implemented. The program demonstrates common methods of the specified collection interfaces, including insertion, removal, searching, sorting, navigation, mapping, and iterator-based traversal.